

ICS – Resumen para el parcial de regularización

CONTENIDO

- | | |
|--|--|
| 1 1 Qué es | 4 10 Niveles y tipos de prueba |
| 1 2 Componentes de un área de proceso ← cae todos los años | 5 1 La distinción |
| 1 3 Los 5 niveles de madurez | 5 2 Metas y prácticas |
| 1 4 Reglas operativas (de acá salen los ejercicios) | 5 3 Grado de confianza |
| 1 5 Madurez vs. capacidad | 5 4 Inyección y remoción de defectos |
| 2 1 Mapa por nivel | 5 5 V&V y los ciclos de vida |
| 2 2 Las tres de gestión de procesos (nivel 3) — el par que más confunden | 6 1 Proyecto |
| 2 3 Las de soporte | 6 2 PP — Planificación (nivel 2) |
| 4 1 Por qué existen | 6 3 PMC — Monitorización y control (nivel 2) |
| 4 2 Cómo elegir la técnica ← la pregunta de correspondencia | 6 4 Los 10 grupos de procesos de gestión |
| 4 3 Particionamiento de equivalencia | 6 5 EDT, precedencias y esfuerzo |
| 4 4 Análisis de valor de frontera | 6 6 Estimación |
| 4 5 Tablas de decisión | 6 7 Tipos de mantenimiento |
| 4 6 Transición de estados | 7 1 Calidad |
| 4 7 Pruebas de casos de uso | 7 2 El software y su ingeniería |
| 4 8 Caja negra vs. caja blanca | 8 1 Estáticas vs. dinámicas |
| 4 9 Los 10 principios de Myers (los que más se preguntan) | 8 2 Formalidad |
| | 8 3 Tipos de revisión |
| | 8 4 Análisis estático |

Condensado de [ICS.md](#) (las 5 unidades), las notas de clase 2026 y el resumen del alumno 2025. El orden **no es el del programa**: va de mayor a menor peso en el parcial, según lo que aparece en [banco-preguntas.md](#).

BLOQUE	PESO EN LOS PARCIALES
1. CMMI: estructura y niveles	alto — aparece todos los años
2. Áreas de proceso: quién hace qué	alto — 6+ preguntas en la serie
3. Activo vs. producto de trabajo	alto — 4-5 puntos de una sola matriz
4. Técnicas de prueba	alto
5. Verificación y Validación	alto (2025 tuvo 4 preguntas)
6. Gestión de proyectos	medio-alto
7. Calidad e Ingeniería de Software	medio
8. Revisiones y técnicas estáticas	medio

1. CMMI — estructura del modelo

1.1 Qué es

CMMI = Modelo de Madurez de Capacidades Integrado. Es una **guía de buenas prácticas**: no dice *cómo* hacer las cosas, dice *qué* hay que lograr. Aporta la experiencia acumulada de otras empresas, un lenguaje común y un punto de partida. Trabajar con un modelo probado da: menos defectos, menos tiempo de entrega, menor costo, más satisfacción del cliente.

Área de proceso = conjunto de prácticas relacionadas que, implementadas conjuntamente, satisfacen un conjunto de objetivos importantes para la mejora en esa área. Son **22**.

1.2 Componentes de un área de proceso ← *cae todos los años*

CATEGORÍA	QUÉ SON	CUÁLES
Requeridos	Lo que la organización debe lograr. Base de las evaluaciones	Metas específicas (SG) y metas genéricas (GG)
Esperados	Lo que puede implementar para lograr lo requerido. Se admiten alternativas propias	Prácticas específicas (SP) y prácticas genéricas (GP)
Informativos	Ayudan a entender cómo aproximarse	Subprácticas · productos de trabajo típicos · ampliaciones · elaboraciones de GP · notas · ejemplos · referencias · declaración de propósito · áreas relacionadas

Las tres frases que hay que tener grabadas:

- Requerido = **sólo las metas**. Las prácticas **no** son obligatorias.
- "Genérico" = la misma declaración se aplica a **varias áreas de proceso**. Las metas y prácticas **genéricas son las que tratan la institucionalización**.
- Numeración: SG n / GG n ; SP x.y donde x = **número de la meta** e y = **secuencia dentro de la meta**.

1.3 Los 5 niveles de madurez

NIVEL	NOMBRE	QUÉ LO CARACTERIZA
1	Inicial	Caos. "Se hace lo que se puede". Hay intentos de normalizar, pero con estrés se abandonan . Gestión del héroe: alguien salva el proyecto a pulmón. El éxito no es repetible
2	Gestionado	Los procesos se planifican y se monitorizan , y se institucionalizan : no se abandonan bajo presión. Hay planes, indicadores, seguimiento. Los procedimientos pueden ser distintos en cada proyecto
3	Definido	Existe un conjunto de procesos estándar de la organización y cada proyecto adapta el suyo desde ahí, según guías de adaptación . Todos trabajan de la misma manera. El rendimiento es predecible sólo cualitativamente
4	Gestionado cuantitativamente	Se gestiona con estadística y datos históricos . Trata las causas especiales de variación. Rendimiento predecible cuantitativamente
5	En optimización	Mejora continua . Trata las causas comunes de variación y cambia el proceso para mejorar su rendimiento

Las comparaciones que preguntan:

- **2 vs 3:** el **alcance de los estándares**. En 2 los procedimientos pueden diferir entre instancias; en 3 se **adaptan del estándar organizacional** y por eso son consistentes.
- **4 vs 5:** el **tipo de variación**. 4 → causas **especiales** + predictibilidad estadística. 5 → causas **comunes** + cambio del proceso.

1.4 Reglas operativas (de acá salen los ejercicios)

1. Para estar en un nivel hay que satisfacer **TODAS** las áreas de ese nivel **y de los anteriores**. Falta una sola → no estás en ese nivel.
2. Los niveles son **acumulativos**: una organización nivel 5 sigue haciendo PP y PMC a diario. Alcanzar nivel 3 **no te exige** de las metas de nivel 2.
3. Se puede instanciar un área de nivel superior sin tener la base, pero eso funciona hasta que aparece el estrés — que es justo cuando más se la necesita.

El ejercicio típico: "cumple con las áreas de nivel 3 y con las de nivel 2 excepto MA" → **nivel 1**. Si falla un área de nivel 2, se cae todo hacia abajo.

1.5 Madurez vs. capacidad

	QUÉ MIDE	CÓMO SE RAZONA
Madurez (representación por etapas)	La organización entera	Conjuntos fijos de áreas por nivel. Es lo que toman en el parcial
Capacidad (representación continua)	Un área de proceso a la vez	La organización elige en qué área quiere crecer

2. Las áreas de proceso — quién hace qué

2.1 Mapa por nivel

NIVEL ÁREAS

2	PP Planificación de proyecto · PMC Monitorización y control · REQM Gestión de requerimientos · MA Medición y análisis · PPQA Aseguramiento de calidad de proceso y producto · CM Gestión de configuración · SAM Gestión de acuerdos con proveedores
3	OPF Enfoque en procesos · OPD Definición de procesos · OT Formación organizativa · RD Desarrollo de requerimientos · TS Solución técnica · PI Integración de producto · VER Verificación · VAL Validación · RSKM Gestión de riesgos · IPM Gestión integrada · DAR Análisis de decisiones
4	OPP Rendimiento de procesos · QPM Gestión cuantitativa
5	OID Innovación y despliegue · CAR Análisis causal y resolución

2.2 Las tres de gestión de procesos (nivel 3) – el par que más confunden

ÁREA	PROPÓSITO EN UNA LÍNEA	PALABRAS QUE LA DELATAN
OPF — Enfoque en procesos	Diagnosticar y planificar la mejora: comprender las fortalezas y debilidades actuales de los procesos de la organización	"fortalezas y debilidades" · "evaluar los procesos" · "comparar con otras organizaciones" · "planes de acción de procesos" · "desplegar"
OPD — Definición de procesos	Definir y mantener los activos: el conjunto de procesos estándar y todo lo que la organización guarda para reutilizar	"repositorio de medición de la organización" · "biblioteca de activos" · "modelos de ciclo de vida" · "guías de adaptación" · "procesos estándar"
OT — Formación organizativa	Que las personas puedan desempeñar sus roles de manera eficaz y eficiente	"formación" · "capacitación" · "habilidades" · "roles"

La regla que resuelve el 90 % de esas preguntas:

OPF diagnostica y despliega. OPD define y guarda. OT capacita. Si la frase dice "**de la organización**" o "**procesos estándar**" y habla de un repositorio o de plantillas → **OPD**, no **MA**.

Prácticas que se preguntan cruzadas (misma lista de opciones, pregunta invertida):

PRÁCTICA	ÁREA
Establecer el repositorio de medición de la organización	OPD
Establecer las descripciones de los modelos de ciclo de vida	OPD
Establecer los criterios y guías de adaptación	OPD
Establecer planes de acción de procesos	OPF
Incorporar las experiencias relativas al proceso en los activos de proceso	OPF
Evaluar los procesos de la organización	OPF

2.3 Las de soporte

REQM (2) — Gestión de requerimientos. Gestiona requerimientos **ya capturados**: documenta los cambios y su razón, y mantiene la **trazabilidad bidireccional** entre requerimientos fuente, de producto y de componentes. SP1.1 Comprensión · SP1.2 Compromiso · **SP1.3 Gestionar los cambios** · SP1.4 Trazabilidad bidireccional · **SP1.5 Identificar inconsistencias** entre el trabajo del proyecto y los requerimientos.

REQM vs RD: RD **captura** (elicitación de necesidades, expectativas y restricciones del cliente). REQM **gestiona** lo ya capturado. **REQM vs CM ante un cambio del cliente:** si todavía se **evalúa cómo afecta** → REQM/SP1.3. Si ya se **decidió aceptarlo** y se tramita → CM/SP2.1 Seguir las peticiones de cambio.

PPQA (2) — Aseguramiento de calidad de proceso y producto. Evalúa **objetivamente** procesos y productos de trabajo contra **las descripciones de proceso, estándares y procedimientos de la organización**.

- La objetividad se logra con **independencia**: un grupo de QA separado del proyecto, o revisiones por pares en culturas abiertas. **Excluye** de evaluar un producto a quien participó en armarlo.
- **No conformidad** = problema que refleja falta de adherencia a estándares, descripciones de proceso o procedimientos. Se trata **primero dentro del proyecto**; si no se resuelve, se **escala** a gerencia. Se sigue hasta su resolución y se **establecen registros**. Se resuelve corrigiendo, **cambiando el estándar incumplido**, u **obteniendo una excepción**.
- **En la práctica:** se designa un **SQA** antes del inicio del proyecto, que planifica las actividades de QA en la **WBS y el calendario**, hace **auditorías periódicas** durante la ejecución, escala lo no resuelto, y al cierre asegura el **informe retrospectivo**. Hitos de auditoría: al final de cada fase, bimensual si la fase se alarga, y revisión aleatoria mensual para mantenimiento correctivo chico.

ATENCIÓN **PPQA vs VER.** Si se contrasta contra **un estándar o procedimiento de la organización** → PPQA. Si se contrasta contra **la especificación del producto** → VER. Que lo haga "el equipo de QA" no lo convierte en PPQA.

CM (2) — Gestión de configuración. Asegura que todo el equipo trabaje sobre la **misma línea base**, da capacidad de **controlar los cambios**, y permite saber **qué se entregó al cliente**. SG1 Establecer líneas base (SP1.1 Identificar elementos · SP1.2 Establecer el sistema · **SP1.3 Crear o liberar líneas base**) · SG2 Seguir y controlar cambios (SP2.1 Seguir las peticiones de cambio · SP2.2 Controlar los elementos) · SG3 Establecer la integridad (SP3.1 Registros · **SP3.2 Auditorías de configuración**). **Desde testing:** CM sirve para versionar los casos de prueba, identificar **qué versión del software se está probando**, y seguir los cambios a los casos. No para diseñar casos nuevos.

MA (2) — Medición y análisis. Mide **el proyecto**.

TIPO	QUÉ EVALÚA	EJEMPLOS
Métricas de proyecto	Progreso, esfuerzo, costo, planificación	% de cumplimiento de hitos · costo de horas por iteración
Métricas de producto	Calidad o desempeño del software	Tasa de defectos en producción · cobertura de pruebas unitarias

RSKM — Gestión de riesgos.

- Orden correcto: **identificar** → **analizar** → **priorizar**. (Falso típico: "el análisis es previo a la identificación".)

- Los riesgos se identifican **desde la planificación**.
- Parámetros: **probabilidad · consecuencia · umbrales**.
- Un riesgo puede ser **aceptado** (documentando la razón) o **vigilado**. Por eso es **falso** que cualquier desvío en un riesgo dispare tratamiento: **depende del umbral**.

3. Activo vs. producto de trabajo ← *la matriz que más puntos vale*

	QUÉ ES	SEÑALES EN EL ENUNCIADO
Activo	Algo que la organización tiene para usar en sus proyectos	plantilla · guía · directriz · modelo · estándar . También las herramientas que adquiere (manual del compilador Java)
Producto de trabajo	El resultado de una tarea de un proyecto concreto	"del cliente" · "del sistema desarrollado" · minuta · código · regla de negocio · descripción del proceso de negocio del cliente

El par que define el criterio:

Plantilla de Manual de Instalación = **activo** (el molde, de la organización). *Manual de Instalación del sistema desarrollado para el cliente* = **producto de trabajo** (lo moldeado, del proyecto).

Ítems ya tomados y su respuesta: ver la [tabla consolidada en el banco](#). En los 10 ítems con clave confirmada, "Ambos" nunca fue la respuesta correcta.

4. Técnicas de prueba

4.1 Por qué existen

No se puede probar **exhaustivamente**: las combinaciones de entradas son inabarcables. Las técnicas sirven para **elegir un subconjunto de casos con alta probabilidad de encontrar defectos**.

Caso de prueba = conjunto de **entradas, condiciones de ejecución y resultados esperados**. Formato mínimo: (**valor de entrada** → **resultado esperado**). Sin resultado esperado **no hay caso de prueba** (principio 1 de Myers).

4.2 Cómo elegir la técnica ← la pregunta de correspondencia

SEÑAL EN EL ENUNCIADO	TÉCNICA
Un campo con rango, longitud o formato; "válidos e inválidos"	Partición de equivalencia / valores límite
"Todas las combinaciones", matriz, reglas de negocio que se cruzan	Tablas de decisión
Secuencia de estados, "pasa de X a Y"	Transición de estados
Flujo básico + alternativos, escenarios punta a punta	Casos de uso
Tengo el código, cobertura de sentencias o decisiones	Caja blanca
Volver a probar lo que andaba, después de un fix	Regresión

4.3 Particionamiento de equivalencia

Agrupar las condiciones de entrada que el sistema **trata igual**: si falla para una, se asume que falla para todas las de esa partición. Se prueba **un representante por partición**.

- Rango "10 a 100" → **1 partición válida + 2 inválidas** (menor y mayor).
- Conjunto discreto "ROJO, BLANCO, NEGRO" → 1 válida + 1 inválida (todos los demás).
- También existen **particiones de salida**: agrupar por resultado.

Formato de tabla que pide la cátedra:

ATRIBUTO	DOMINIO	VÁLIDAS	INVÁLIDAS
Día del mes	Entero positivo entre 1 y 31	PV1) $1 \leq X \leq 10$ · PV2) $11 \leq X \leq 20$ · PV3) $21 \leq X \leq 31$	PI1) letras · PI2) $X \leq 0$ · PI3) $X > 31$ · PI4) vacío · PI5) imagen · PI6) carácter especial · PI7) cadena

Y después la tabla de casos: **Caso | Partición | Entrada | Salida esperada**.

ATENCIÓN Los dos errores que se cobran caro. (a) **La zona muerta cuenta**. "Se activa si sube por encima de 25" y "se apaga si cae por debajo de 20" dejan **tres** particiones: ≤ 19 , **20–25 (no pasa nada)**, ≥ 26 . Olvidar la del medio es el error que se repitió en los dos termómetros del parcial. (b) **Las particiones no se pisan**. Si PV3 es 21–31, la inválida es $X > 31$, no $X \geq 31$.

4.4 Análisis de valor de frontera

Mejora al particionamiento: en vez de un representante por partición, prueba los **extremos**, que es donde se agrupan los errores.

- Rango 10–100 → probar **9, 10, 100 y 101**.
- Conjuntos ordenados → primer y último elemento.
- Minutos → siempre **0 y 59**. Fechas → incluir meses, **años bisiestos y no bisiestos**.
- **No numéricos**: una letra cualquiera → 1 caso · conjunto cerrado de N valores → N casos · cadena de longitud fija → esa longitud exacta · hasta N caracteres → N-1, N, N+1.

La diferencia que preguntan: partición de equivalencia = **un caso por partición**; valores límite = **varios casos por partición**, en los bordes. Si la consigna pide "cubrir todas las particiones válidas", va **un representante de cada una**, no los bordes.

4.5 Tablas de decisión

Para cuando **múltiples combinaciones de entradas** producen resultados distintos. Se centra en **la lógica y las reglas de negocio**: filas de condición + filas de acción, y **cada columna es una regla**.

Cómo se cuenta (es lo que preguntan):

1. **Condiciones** = las preguntas binarias o los atributos que deciden. Una condición puede tener **más de dos valores** (categoría Coulson/IronMan/Hulk = 1 condición, 3 valores).
2. **Acciones** = los resultados distintos que el sistema puede ejecutar.
3. **Columnas sin reducir** = producto de los valores posibles de cada condición.
4. **Columnas tras reducir** = se eliminan las **combinaciones imposibles** (si el usuario no existe, la contraseña es irrelevante) y se colapsan las indiferentes.

Ejemplo resuelto (gimnasio, parcial 2024): categorías Coulson/IronMan(10 %)/Hulk(15 %) + cuota al día (5 % adicional, acumulable). → **4 acciones** (0 %, 10 %, 15 %, +5 %) · **6 columnas** (3 categorías × 2 estados de cuota) · **6 casos válidos**.

4.6 Transición de estados

Para sistemas modelables como **máquina de estados finitos**, donde la salida ante la misma entrada **depende del estado anterior**. Permite detectar incompatibilidades: falta de transición, o estados de los que no se puede salir. Una prueba completa incluye **transiciones no válidas** (intentos fallidos, timeouts) y **eventos no especificados** (cancelar).

4.7 Pruebas de casos de uso

Ejercitan el sistema **de punta a punta**. Proceso: definir el **flujo básico** (camino feliz) y los **flujos alternativos** → **derivar escenarios** (cada escenario = el flujo básico más uno de los caminos alternativos) → **un caso de prueba por escenario**. Son de **mejor nivel** que los casos armados a mano: validar una tarjeta en un cajero con-templa mucho más que meter tres números al azar.

4.8 Caja negra vs. caja blanca

	QUÉ TENGO	VENTAJA / DESVENTAJA
Caja negra	Sólo entrada y salida	Rápida, no requiere el código. No sé dónde está el error
Caja blanca	El código fuente	Sé dónde está el problema; puedo medir cobertura. Requiere acceso al código

Técnicas de caja blanca, sobre **secuencia, selección e iteración**:

TÉCNICA	OBJETIVO
Sentencia	Ejecutar cada sentencia al menos una vez
Decisión	Evaluar cada decisión en verdadero y falso
Caminos	Cada camino de ejecución independiente. No prueba todas las combinaciones

Un defecto puede aparecer **aunque todas las sentencias se hayan ejecutado**, porque surge al **combinarse** ciertos caminos.

4.9 Los 10 principios de Myers (los que más se preguntan)

1. Una parte **necesaria** del caso de prueba es la **salida prevista**.
2. El desarrollador **no debe probar su propio programa**.
3. El personal de prueba **no debería depender** del área de desarrollo.
4. **Inspeccionar los resultados** de la prueba.
5. Escribir casos para las condiciones **esperadas y las no esperadas**.
6. Comprobar que el programa **no hace lo que no debe**.
7. Evitar casos **desechables y sin documentar** → guardarlos para **regresión**.
8. No planificar las pruebas **suponiendo que no habrá errores**.
9. La probabilidad de encontrar errores en una sección es **proporcional a los ya encontrados** ahí → conviene concentrar el esfuerzo en los módulos con más defectos.
10. Probar es una tarea **altamente creativa**.

Por qué el que desarrolla no prueba: desarrollar es **creativo**, probar es **destrutivo** · **tunnel vision** (entiende su desarrollo de raíz y prueba lo que ya sabe que anda) · si el error está en **cómo entendió el requerimiento**, va a probar según su propio malentendido.

4.10 Niveles y tipos de prueba

NIVEL	QUÉ PRUEBA	QUIÉN
Unitarias	Cada módulo aislado	El propio desarrollador ; usa stubs y drivers
Integración	Interacción entre módulos; defectos de interconexión	Referencia: el diseño
Sistema	Comportamiento global contra la especificación (funcionales y no funcionales)	Equipo independiente ; caja negra; entorno similar a producción
Aceptación	Que satisface las necesidades del usuario	El usuario o cliente

Ningún nivel reemplaza a otro. En un ciclo incremental se hacen **todos los niveles en todos los incrementos**, no sólo en el último.

Estrategias de integración: **big-bang** (todo junto; descubre los problemas al final) · **bottom-up** (necesita **drivers**; no detecta problemas de diseño hasta tarde) · **top-down** (necesita **stubs**; **descubre rápido los errores de arquitectura**).

Aceptación — modalidades: **alfa** (clientes acotados, entorno controlado) · **beta** (clientes más amplios) · **piloto** (pocos departamentos, en **producción** del cliente).

Regresión vs. confirmación. La **confirmación** verifica que el defecto corregido realmente se solucionó. La **regresión** verifica que el arreglo no rompió otra cosa.

5. Verificación y Validación

5.1 La distinción

VERIFICACIÓN (VER, NIVEL 3)	VALIDACIÓN (VAL, NIVEL 3)
Los productos de trabajo cumplen sus requerimientos especificados	El producto se ajusta a su uso previsto en su entorno previsto
¿Estoy construyendo <i>correctamente</i> el producto?	¿Estoy construyendo el producto <i>correcto</i> ?
Cuestión interna : contra la especificación	Involucra al cliente / usuario

El criterio que resuelve todas las preguntas: **mirá contra qué se contrasta, no quién lo hace ni en qué fase.**
Contra la especificación → VER. Contra la necesidad del usuario → VAL. Pruebas de **sistema** = VER (contra especificación). Pruebas de **aceptación** = VAL. Inspecciones y revisiones = **VER**.

5.2 Metas y prácticas

VER	VAL
SG1 Preparar la verificación: SP1.1 Seleccionar productos · SP1.2 Establecer el entorno · SP1.3 Procedimientos y criterios	SG1 Preparar la validación: SP1.1 Seleccionar productos · SP1.2 Establecer el entorno · SP1.3 Procedimientos y criterios
SG2 Realizar revisiones entre pares: SP2.1 Preparar · SP2.2 Llevar a cabo · SP2.3 Analizar los datos	(VAL no tiene revisiones entre pares)
SG3 Verificar los productos seleccionados: SP3.1 Realizar la verificación · SP3.2 Analizar resultados	SG2 Validar el producto: SP2.1 Realizar la validación · SP2.2 Analizar resultados

ATENCIÓN "Revisiones entre pares" existe **SÓLO** en VER. Si una opción dice VAL / SP2.1 Preparar las revisiones entre pares, la respuesta es **NINGUNA**.

5.3 Grado de confianza

V&V no busca ausencia total de defectos, sino que el software sea **suficientemente bueno para su uso previsto**. Depende de:

- **Criticidad del sistema** — crítico → confianza alta; prototipo → menor.
- **Expectativas del usuario** — la tolerancia a fallos viene bajando.
- **Entorno de mercado** — con **pocos competidores** se puede lanzar antes de estar completamente probado; con **precio bajo**, los clientes toleran más defectos.

5.4 Inyección y remoción de defectos

Los defectos **se inyectan en todas las fases** (plan, análisis, diseño, construcción, implantación), no sólo al programar. V&V es la **remoción**: las **revisiones** cubren desde el plan hasta la construcción, las **pruebas** desde la construcción hasta la implantación. **Corregir un defecto en mantenimiento cuesta ~100 veces más que en requisitos.**

5.5 V&V y los ciclos de vida

CICLO	RELACIÓN CON V&V	CUÁNDO CONVIENE
Cascada	Las pruebas van al final → detección tardía y cara	Requerimientos conocidos y estables ; mantenimiento correctivo corto. El que menos sirve con requerimientos inestables
Modelo en V	Integra V&V desde las primeras fases , en paralelo. Cada etapa de desarrollo tiene su nivel de prueba	—
Incremental	Cada incremento es una mini-cascada completa (análisis, diseño, código, prueba). Se prueba a medida que se agrega	Cuando hay que salir antes a producción
Iterativo	Pasadas sucesivas, más flexible que el incremental	Requerimientos no estabilizados
Espiral	Prototipo inicial + prueba, rediseño y prototipado continuos . Riesgo: entregar un prototipo como producto final	Cuando no se puede especificar por adelantado
Prototipado	Diseño rápido de lo visible → evaluación del cliente → refinamiento	Objetivos generales sin requisitos detallados. Reduce el riesgo de subestimar el esfuerzo

Verificaciones sobre el documento de requisitos: validez (que sean las funciones necesarias) · **consistencia** (que no se contradigan) · **completitud** (que estén todas) · **realismo** (implementables con el presupuesto y la planificación) · **verificabilidad** (que se pueda construir una prueba que lo demuestre).

6. Gestión de proyectos

6.1 Proyecto

Conjunto de actividades coordinadas y controladas, con inicio y fin definidos, que crea un producto o servicio **único** conforme a requisitos específicos, dentro de límites de tiempo, coste y recursos. Se desarrolla **en pasos** (elaboración gradual).

- Un proyecto **siempre tiene fin**. (Falso típico: "un proyecto puede no tener fin".)
- Los proyectos de software **no son sólo de desarrollo**: hay **DES** (desarrollo), **MANT** (mantenimiento) y **DESPL** (despliegue/implantación).
- Tener un **Gantt no es tener un plan de proyecto**: el Gantt es una vista del cronograma; el plan incluye alcance, estimaciones, recursos, riesgos, calidad y comunicación.

6.2 PP – Planificación (nivel 2)

Desarrollar el plan · interactuar con las partes interesadas · **obtener el compromiso** · mantener el plan. Arranca **con los requerimientos**.

Incluye: estimación de las tareas · determinación de los recursos · **identificación de riesgos** · presupuesto y calendario · definir el ciclo de vida. **No incluye**: el pago a los recursos, ni la especificación detallada de la arquitectura (eso es TS).

"Definir el equipo del proyecto" = determinar **roles y responsabilidades**. No incluye estimar el esfuerzo por rol ni controlar el cronograma.

6.3 PMC – Monitorización y control (nivel 2)

Comprender el progreso para tomar **acciones correctivas** cuando el rendimiento **se desvía significativamente** del plan. Se compara calidad, esfuerzo, coste y calendario **reales contra el plan**, en los **hitos definidos en la EDT/WBS**.

Acciones correctivas: replanificación · nuevos acuerdos · actividades adicionales de mitigación dentro del plan actual.

Desvío significativo = el que, **si se deja sin resolver, impide al proyecto cumplir sus objetivos**. Por eso es **falso** que "cualquier desvío debe ser resuelto".

PP vs PMC — la regla del parcial. Mirá dónde está la reunión respecto del momento narrado: reunión **en el futuro** ("para informarlo en la reunión donde se juntará por primera vez todo el equipo") → **PP**. Reunión **ya ocurrida** ("en la última reunión de avance...") → **PMC**.

Los controles y puntos de monitoreo se definen en la fase de **PLANIFICACIÓN**, no en ejecución.

6.4 Los 10 grupos de procesos de gestión

GRUPO	PROCESOS
Coordinación	Iniciar el proyecto · Desarrollar el plan · Gestionar la ejecución · Supervisar el trabajo · Control integrado de cambios · Cerrar
Alcance	Definir el alcance · Definir las actividades · Controlar y verificar el alcance
Tiempo	Establecer la secuencia de actividades · Estimar la duración · Desarrollar el cronograma · Controlar el cronograma
Costes	Estimar los costos · Elaborar los presupuestos · Controlar los costos

GRUPO	PROCESOS
Calidad	Planificar · Aseguramiento · Control
Recursos	Planificar · Controlar
Personal	Definir el equipo · Gestionar el equipo
Comunicación	Planificar · Gestionar la información y los interesados
Riesgos	Planificar · Identificar · Analizar · Planificar la respuesta · Controlar
Adquisiciones	Planificar · Contratar · Solicitar y seleccionar proveedores · Administrar y cerrar el contrato

El distractor clásico: "*Definir las actividades*" suena a cronograma pero pertenece a **Alcance**, no a Tiempo.

Detalles que preguntan: **desarrollar el cronograma** debe identificar explícitamente el **camino crítico** · **elaborar los presupuestos** incluye las **reservas para contingencias**.

6.5 EDT, precedencias y esfuerzo

Secuencia: EDT → tareas típicas del ciclo de vida → **órdenes de precedencia**.

- **Predecesora directa** = la inmediatamente anterior. Si A precede a B y B precede a C, **A no es predecesora directa** de C.
- **Módulos independientes arrancan en paralelo**: con módulos X, Y, Z, las tareas de inicio son **Analizar X, Analizar Y y Analizar Z** (tres, no una).

ATENCIÓN **Esfuerzo ≠ duración**. El **esfuerzo se suma** (días-persona, horas-persona); la **duración no**, porque las tareas de módulos independientes corren en paralelo. Una tarea puede durar 2 días y consumir 3 días-persona. Si el ejercicio pide **esfuerzo**, sumá la columna esfuerzo y convertí (1 día-persona = 8 h).

6.6 Estimación

MÉTODO	CÓMO
Valor esperado ("3 puntos")	Estimar cada tarea en optimista, normal y pesimista y combinarlas. Fórmula estándar: $(O + 4N + P) / 6$
Delphi	Estimación grupal e independiente de expertos, iterativa , buscando convergencia
Puntos de función	Medida indirecta del tamaño , independiente de la tecnología
Puntos de historia	Estimación relativa, de metodologías ágiles

Distribución 40-20-40: análisis y diseño **40 %** (10-15 análisis, 25-30 diseño) · codificación **20 %** · pruebas **40 %**. La lectura: **codificar es la quinta parte del proyecto**.

APF en tres pasos:

1. **Identificar componentes** tras fijar el **límite del sistema**: entradas · salidas · consultas · **ficheros lógicos internos (FLI)** · **ficheros de interfaz externos (FIE)**.
2. **Ponderar** por complejidad baja/media/alta:

	BAJA	MEDIA	ALTA
Entradas	3	4	6
Salidas	4	5	7
Consultas	3	4	6
FLI	7	10	15
FIE	5	7	10

La suma = **PFD** (puntos función sin ajustar).

3. **Ajustar** con 14 características valoradas de 0 a 5. La suma es el **TDI** (0–70):

Factor de ajuste = $0,65 + (\text{TDI} / 100)$ → va de 0,65 a 1,35
 PF ajustados = PFD × Factor de ajuste

6.7 Tipos de mantenimiento

TIPO	OBJETIVO
Correctivo	Corregir errores detectados
Evolutivo / perfectivo	Incorporaciones, modificaciones y eliminaciones para cubrir el cambio en las necesidades del usuario
Adaptativo	Modificaciones por cambios en el entorno (hardware, software de base, gestores de BD, comunicaciones)
Preventivo	Mejorar la calidad interna y la mantenibilidad, sin cambio funcional visible

7. Calidad e Ingeniería de Software

7.1 Calidad

Calidad = idoneidad de uso. Tres definiciones que conviven:

- **Juran / clásica**: características del producto que satisfacen las necesidades del cliente + **inexistencia de deficiencias**.
- **CMMI**: capacidad de un conjunto de características inherentes de un producto, componente o **proceso** de satisfacer por completo los requisitos del cliente.
- **ISO**: propiedades y características que le confieren aptitud para satisfacer necesidades **explícitas o implícitas**.

Elementos que influyen: procesos y buenas prácticas · herramientas · personas · medidas y métricas.

Tres niveles de gestión de la calidad: **producto** (pruebas en paralelo a cada etapa) · **proyecto** (controlar fases y áreas de gestión) · **proceso** (gestionar las áreas de proceso de toda la organización con una metodología → acá juega CMMI).

Tipos de calidad de producto: **interna** (no la percibe el cliente: mantenibilidad, claridad del código, estándares, reusabilidad) · **externa** (cumplimiento de requerimientos) · **de uso** ("calidad futura": la que aparece recién al usarlo de verdad).

Dimensiones críticas sobre las que las organizaciones se centran para mejorar: personas · métodos y procedimientos · herramientas y equipamiento.

7.2 El software y su ingeniería

Software = programas + datos + documentos. Es un elemento **lógico**.

Características que obligan a tratarlo distinto:

- Se **desarrolla, no se fabrica**: no hay línea de producción, cada producto es único.
- El recurso principal son **las personas**, y **no son intercambiables con el tiempo**: sumar gente no acelera linealmente (coordinación, comunicación, curva de aprendizaje).
- **No se estropea, se deteriora**: cada cambio de mantenimiento puede introducir defectos.
- La **reutilización** está lejos de su potencial, justamente porque cada producto es único.

Ingeniería del software (IEEE): aplicación de un enfoque **sistemático, disciplinado y cuantificable** al desarrollo, operación y mantenimiento del software.

Tecnología multicapa: sobre una base de **compromiso con la calidad** se apoyan **Procesos → Métodos → Herramientas**.

Etapas: análisis de requisitos → especificación → diseño y arquitectura → programación → **prueba** → mantenimiento. *La etapa que comprueba que el software realiza correctamente lo indicado en la especificación es **Prueba**.*

Principios de la Ingeniería de Software que preguntan:

-  **Haz de la calidad la razón de trabajar**
 -  **Probar, probar y probar**
 -  "Las personas y el tiempo son intercambiables"
 -  "Primero hazlo rápido, luego hazlo correcto"
 -  "Forzar el mismo modelo de ciclo de vida en todos los proyectos"
-

8. Revisiones y técnicas estáticas

8.1 Estáticas vs. dinámicas

ESTÁTICAS (REVISIONES)		DINÁMICAS (PRUEBAS)
Qué buscan	Defectos	Fallos
Requieren	Documentos (requisitos, diseño, código)	El sistema andando
Cuándo	Cualquier momento del ciclo	Sólo con el producto ejecutable
Ventaja	Sé dónde está el problema; encuentro varios a la vez; corrijo temprano	Más rápidas
Desventaja	Más lentas	No sé dónde está el error (caja negra)

Son **complementarias**. Las revisiones están vinculadas principalmente a **verificación**, y son la **primera forma de prueba aplicable** en el ciclo de vida.

8.2 Formalidad

INFORMALES	FORMALES
No hay proceso definido · no hay roles · usualmente no planeadas	Objetivos definidos · proceso documentado · roles definidos · checklists · reporte de resultados · recolección de datos para el control del proceso

La formalidad importa porque deja **trazabilidad documentada** de decisiones.

8.3 Tipos de revisión

TIPO	QUIÉN DIRIGE	FOCO	FORMALIDAD
Informal	—	Encontrar defectos; documentar es opcional	Mínima
Walkthrough	El propio autor	Entendimiento común, discutir alternativas	Media
Revisión técnica	Moderador o experto	Consenso técnico , no búsqueda de defectos	Variable
Revisión entre pares	Colegas del mismo proyecto	Eliminar defectos temprano	Media
Inspección	Moderador formado, NO el autor	Registrar defectos (las discusiones se posponen); criterios de salida	Máxima

Los 5 roles: **moderador** (dirige, hace validación de entrada y seguimiento) · **autor** · **documentador** (anota defectos y sugerencias) · **revisor** (valida el material **antes** de la reunión) · **supervisor** (decide destinar tiempo del proyecto a las revisiones).

Proceso básico: identificar los entregables → armar la lista de participantes → los revisores **estudian** el documento → identifican problemas y los comunican al autor → el autor responde y actualiza.

Guías para revisiones entre pares: entorno **seguro y no amenazante** · capacitar en los roles · **documentar los defectos** · **enfocarse en el producto, no en la persona** · **incluirlas en la planificación** para que tengan tiempo asignado.

Las organizaciones tienden a **sobreestimar el costo de las revisiones y subestimar sus beneficios**, y por eso a veces no las implementan.

8.4 Análisis estático

Busca defectos **sin ejecutar**, pero **una vez escrito el código**, con analizadores automáticos. Detecta: variables **no inicializadas** · variables **no utilizadas** · **código inalcanzable** · inconsistencias entre módulos · vulnerabilidades · violaciones de estándares. Métrica clave: **complejidad ciclomática** = n° de sentencias de decisión binarias + 1; sirve para **estimar cuántas pruebas** necesita un componente.

9. Errores que se repiten en los parciales

1. Olvidar la zona muerta al partir en clases de equivalencia.
2. Confundir partición con valor límite — "cubrir las particiones válidas" pide representantes, no bordes.
3. Sumar duración en vez de esfuerzo.
4. Marcar "Ambos" en la matriz de activo/producto de trabajo.
5. Creer que las prácticas son obligatorias — sólo las metas lo son.
6. Creer que alcanzar nivel 3 exime de las metas de nivel 2.
7. Confundir MA con OPD cuando dice "repositorio de la organización".
8. Mandar a VAL lo que se contrasta contra la especificación (pruebas de sistema = VER).
9. Poner "revisiones entre pares" en VAL — sólo existen en VER.
10. Resolver cualquier desvío — sólo los significativos.
11. Contar una condición de 3 valores como binaria en tablas de decisión.
12. Olvidar que los módulos independientes arrancan en paralelo (tres tareas de inicio, no una).